```
class Program
{
    static void Main(string[] args)
    {
        Console.WriteLine("=== ТЕСТИРОВАНИЕ ФУНКЦИЙ АКТИВАЦИИ (OPENCL JIT)
===");
        try
        {
            // 1. Инициализация OpenCL
            using var computeContext = new ComputeContext(preferAccelerator:
true);
            using var queue =
computeContext.Context.CreateCommandQueue(computeContext.Device);
            DeviceQueueRegistry.RegisterCurrent(queue);
            Console.WriteLine($"Выбран девайс: {computeContext.Device.Name}");

            // Геометрия: Батч = 2 примера, Входных фичей = 3, Выходных фичей =
4
            int batch = 2;
            int inFeatures = 3;
            int outFeatures = 4;

            float[] inputData = new float[] {
                1.0f, 2.0f, 3.0f, // Пример 1
                4.0f, 5.0f, 6.0f // Пример 2
            };

            float[] targetData = new float[] {
                0.0f, 1.0f, 0.0f, 0.0f, // Желаемый выход для Примера 1
                1.0f, 0.0f, 0.0f, 0.0f // Желаемый выход для Примера 2
            };

            // Инициализация тензоров данных
            using var xTensor = new Tensor(inputData, new[] { batch, 1,
inFeatures }, computeContext.Context, MemoryFlags.ReadWrite, requiresGrad:
true);
            using var targetTensor = new Tensor(targetData, new[] { batch, 1,
outFeatures }, computeContext.Context, MemoryFlags.ReadWrite, requiresGrad:
false);

            var nnClGlob = new NnClGlob(computeContext);

            // 2. Создание нашего нового слоя Linear!
            using var linearLayer = new Linear(computeContext, nnClGlob,
inFeatures, outFeatures);

            // Передаем параметры слоя в оптимизатор SGD
            var optimizer = new SGD(linearLayer.GetParameters(), learningRate:
0.2f, computeContext);

            Console.WriteLine("\n--- Выполнение шага обучения слоя Linear ---");
            for (int epoch = 0; epoch < 10; epoch++)
            {
                Console.WriteLine($"эпоха {epoch + 1}!");

                GradientStorage.Clear();
                linearLayer.ZeroGrad(); // Сбрасываем старые буферы градиентов
параметров

                // A. Прямой проход слоя (BMM + JIT Bias)
                Tensor pred = linearLayer.Forward(xTensor);
                Console.WriteLine($"[Forward] Выходной тензор успешно рассчитан.
Форма: [{string.Join(", ", pred.Shape)}]");
```

```
                // Б. Расчет величины ошибки
                Tensor loss = nnClGlob.MseLoss(pred, targetTensor);
                float[] lossCpu = HostReturner.ToCpu(loss);
                Console.WriteLine($"[Loss] Текущая ошибка слоя:
{lossCpu[0]:F6}");

                // В. ЗАПУСК ОБРАТНОГО ПРОХОДА (Autograd)
                // Градиент пойдет: MSE -> LocalOp (дельта по bias) -> BmmOp ->
Веса Weight -> Входной тензор X
                AutoGrad.Backward(loss, computeContext.Context);
                Console.WriteLine("[Backward] Каскадный обратный проход графа
успешно завершен!");

                // Г. Проверка сцепки буферов градиентов на GPU во VRAM
                Console.WriteLine("\nАнализ буферов градиентов после сцепки:");
                Console.WriteLine($" -> Градиент для смещения Bias создан:
{linearLayer.Bias.Grad != null}");
                Console.WriteLine($" -> Градиент для матрицы весов Weight
создан: {linearLayer.Weight.Grad != null}");
                Console.WriteLine($" -> Градиент дотек до самого первого входа
X: {xTensor.Grad != null}");

                if (linearLayer.Bias.Grad != null && linearLayer.Weight.Grad !=
null)
                {
                    float[] biasGrad = HostReturner.ToCpuGrad(linearLayer.Bias);
                    float[] weightGrad =
HostReturner.ToCpuGrad(linearLayer.Weight);
                    Console.ForegroundColor = ConsoleColor.Green;
                    Console.WriteLine($"\n[Успех] Градиент Bias (все 4 эл.):
[{string.Join(", ", biasGrad.Select(x => x.ToString("F4")))}]");
                    Console.WriteLine($" Градиент весов Weight (первые 4):
{weightGrad[0]:F5}, {weightGrad[1]:F5}, {weightGrad[2]:F5},
{weightGrad[3]:F5}");
                    Console.ResetColor();

                    // Д. Физическое обновление параметров во VRAM
                    optimizer.Step();
                    Console.WriteLine("\n[SGD] Шаг оптимизации успешно применен.
Веса слоя Linear обновлены на GPU.");
                }
                else
                {
                    Console.ForegroundColor = ConsoleColor.Red;
                    Console.WriteLine("\n[Ошибка] Градиенты застряли на стыке
слоев!");
                    Console.ResetColor();
                }
            }

            Console.WriteLine("\n=== ТЕСТ ОБУЧАЕМОГО СЛОЯ LINEAR С УСПЕХОМ
ЗАВЕРШЕН! ===");
        }
        catch (Exception ex)
        {
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"\n[КРИТИЧЕСКАЯ ОШИБКА]: {ex.Message}");
            Console.ResetColor();
        }
        finally
        {
            DeviceQueueRegistry.Clear();
        }
        Console.ReadLine();
```

```
    }
}
```

Исправь пожалуйста этот пример так, чтобы он содержал функцции активации. (тоесть был компонент localRunner)/

Linear не содержит функций активации которые были реализованны:

```
using cstorchcl.compiler;
using NOpenCL;
using System;
using System.Collections.Generic;

namespace cstorchcl.core
{
    /// <summary>
    /// Полносвязный (линейный) слой нейросети с поддержкой JIT-сцепки и
Broadcasting смещения на GPU.
    /// </summary>
    public class Linear : NeuralNetwork
    {
        /// <summary>
        /// Обучаемая матрица весов слоя. Форма: [1, InFeatures, OutFeatures]
        /// </summary>
        public Tensor Weight { get; private set; }

        /// <summary>
        /// Обучаемый вектор смещения (bias). Форма: [1, OutFeatures] — будет
автоматически растягиваться по батчу.
        /// </summary>
        public Tensor Bias { get; private set; }

        private readonly int _inFeatures;
        private readonly int _outFeatures;

        /// <summary>
        /// Инициализирует линейный слой и заполняет его веса случайным шумом
для разрушения симметрии.
        /// </summary>
        public Linear(ComputeContext computeContext, NnClGlob nnClGlob, int
inFeatures, int outFeatures)
            : base(computeContext, nnClGlob)
        {
            _inFeatures = inFeatures;
            _outFeatures = outFeatures;

            // 1. Аллокация памяти во VRAM девайса
            // Веса делаем 3D формы [1, K, N] под требования нашего пакетного
ядра BMM
            Weight = RegisterParameter(new[] { 1, inFeatures, outFeatures },
requiresGrad: true);
            // Bias делаем 2D формы [1, OutFeatures] под требования Broadcasting
            Bias = RegisterParameter(new[] { 1, outFeatures }, requiresGrad:
true);

            // 2. Инициализация весов методом Ксавье (Xavier/Glorot
Initialization)
            // Это стандарт глубокого обучения, предохраняющий tanh от насыщения
и зануления градиентов
            CommandQueue queue = DeviceQueueRegistry.GetCurrent();
```

```
            var rand = new Random();

            float bound = (float)(Math.Sqrt(6.0) / Math.Sqrt(inFeatures +
outFeatures));

            // Заполнение матрицы весов W
            float[] hostWeights = new float[Weight.Size];
            for (int i = 0; i < hostWeights.Length; i++)
            {
                hostWeights[i] = (float)(rand.NextDouble() * 2.0 - 1.0) * bound;
            }
            long weightsBytes = (long)Weight.Size * sizeof(float);
            unsafe
            {
                fixed (float* hw = hostWeights)
                {
 queue.EnqueueWriteBuffer(Weight.DataBuffer, true, 0, weightsBytes, (IntPtr)hw);
                }
            }


            // Заполнение вектора смещения Bias (по умолчанию заполняем нулями,
как в PyTorch)
            float[] hostBias = new float[Bias.Size];
            long biasBytes = (long)Bias.Size * sizeof(float);
            unsafe
            {
                fixed (float* hb = hostBias)
                {
queue.EnqueueWriteBuffer(Bias.DataBuffer, true, 0, biasBytes, (IntPtr)hb);
                }
            }


            queue.Finish();
        }

        /// <summary>
        /// Выполняет высокооптимизированный прямой проход: BMM -> JIT Сложение
с Bias.
        /// На вход ожидает 3D тензор данных формы [Batch, M, InFeatures].
        /// </summary>
        public Tensor Forward(Tensor x)
        {
            if (x.Shape[x.Shape.Length - 1] != _inFeatures)
            {
                throw new ArgumentException($"Несоответствие размерностей входа.
Ожидалось {_inFeatures} признаков, получено {x.Shape[x.Shape.Length - 1]}.");
            }

            // ШАГ 1 (ГЛОБАЛЬНЫЙ): Тяжелое матричное умножение X x W на GPU
            // Выход projected получит форму [Batch, M, OutFeatures]
            Tensor projected = NnClGlob.Bmm(x, Weight);

            // ШАГ 2 (ЛОКАЛЬНЫЙ JIT): Сплавление операции прибавления Bias на
регистрах GPU.
            // Передаем projected и bias в наш Roslyn-компонент
            var tensorArgs = new Dictionary<string, Tensor> {
                { "P", projected },
                { "b", Bias }
            };

            // Математический псевдокод для JIT.
```

```
            // Благодаря нашему модулю Broadcasting, вектор 'b' автоматически
прибавится к каждой строке матрицы 'P'!
            string pseudoCode = "P = P + b;";

            // Вызываем RunLocal, который вернет НОВЫЙ чистый тензор без
затирания графа Autograd
            Dictionary<string, Tensor> localOutputs =
NnClGlob.RunLocal(pseudoCode, tensorArgs, constants: null);
            Tensor output = localOutputs["P"];

            return output;
        }
    }
}
```

Вот примерн реализщации запуска фугкций активации:

```
// Создаем символические обертки для LocalRunner
var A = new LocalVariable(tensorA, "A");
var B = new LocalVariable(tensorB, "B");
var C = new LocalVariable(tensorC, "C");

float q = 1.5f;
uint seed = 42;
int iterations = 1000;

// 1. Создаем словарь связей строковых имен и физических тензоров GPU
var tensorArgs = new Dictionary<string, Tensor> {
    { "A", tensorA },
    { "B", tensorB },
    { "C", tensorC }
};

// 2. Создаем словарь захваченных внешних констант C#
var constants = new Dictionary<string, float> {
    { "q", 1.5f }
};

// 3. Описываем монолитную пачку математики в виде простого текстового блока
(Псевдокод)
string codeBlock = @"
    A = nnclloc.tanh(A);
    C = nnclloc.cos(B);
    A = A / q;
    C = nnclloc.dropout(C, 0.2, 42);
    B = B + 0.01;
";

Console.WriteLine("Тест 2: Запуск 1000 итераций через промышленный Roslyn JIT-
Fusion...");

Stopwatch swFused = Stopwatch.StartNew();
for (int i = 0; i < iterations; i++)
{
    // Запуск конвейера. Roslyn мгновенно соберет код ядра,
    // а JITProgramCache выдаст уже скомпилированный бинарник со 2-й итерации!
    NnClLoc.Run(codeBlock, tensorArgs,computeContext, constants);
}
queue.Finish();
```

А вот список поддерживаемых локальных функций активации:

```
 public static class nnclloc
 {
     public static LocalVariable sin(LocalVariable v) => v;
     public static LocalVariable cos(LocalVariable v) => v;
     public static LocalVariable tanh(LocalVariable v) => v;
     public static LocalVariable dropout(LocalVariable v, float p, uint seed) =>
v;
     public static LocalVariable relu(LocalVariable v) => v;
     public static LocalVariable softplus(LocalVariable v) => v;
     public static LocalVariable leakyrelu(LocalVariable v) => v;

 }
```

Также испаользуй глобальные компоненты:

```
 /// <summary>
 /// Глобальная безопасная операция Softmax (Forward).
 /// Рассчитана на работу со скорами внимания размерности [B * H * L, SEQ_LEN].
 /// </summary>
 public Tensor Softmax(Tensor x, int dim = -1)
 {
    // Эквивалент проверки размерности в Python: assert dim == -1 or dim == len(x.shape) - 1
    int targetDim = dim == -1 ? x.Shape.Length - 1 : dim;
    if (targetDim != x.Shape.Length - 1)
    {
       throw new NotSupportedException("Текущая реализация ядра Softmax поддерживает
редукцию только по последней оси.");
    }

    int seqLen = x.Shape[x.Shape.Length - 1]; // Последняя ось
    int numRows = x.Size / seqLen;           // Вычисляем количество строк

    // Создаем выходной тензор той же формы во VRAM (RequiresGrad пока передаем как у
исходного)
    Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad:
x.RequiresGrad);

    // Настройка сетки потоков (Сетка идентична вашей конфигурации на Python)
    IntPtr localSize = new IntPtr(256);
    IntPtr globalSize = new IntPtr(numRows * 256);

    // Получаем глобальную активную очередь выполнения задач из нашего реестра
    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

    // Передаем аргументы в Си-ядро по порядку индексов (0, 1, 2, 3):
    // kernel void softmax_forward(global const float* X, global float* Y, const int seq_len)
    _kernelSoftmaxForward.Arguments[0].SetValue(x.DataBuffer);
    _kernelSoftmaxForward.Arguments[1].SetValue(y.DataBuffer);
    _kernelSoftmaxForward.Arguments[2].SetValue(seqLen);

    // Запуск выполнения ядра на GPU
```

```
    activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxForward, null, new[] { globalSize },
new[] { localSize });

    // Синхронизация: гарантирует завершение вычислений перед чтением
    activeQueue.Finish();

    // Если включен режим Autograd и требуется расчет градиента, привязываем контекст
создателя к тензору
    if (y.RequiresGrad)
    {
        // Создаем операцию, передавая входы и выходы
        y.Creator = new SoftmaxOp(x, y, this);
    }

    return y;
 }
 /// <summary>
 /// Обратный проход глобальной операции Softmax.
 /// Использует уже вычисленный forward-выход тензора Y для экономии памяти VRAM.
 /// </summary>
 /// <param name="gradOutput">Пришедший сверху градиент dL/dY [B * H * L,
SEQ_LEN]</param>
 /// <param name="y">Исходный выход прямого прохода Softmax (вероятности) [B * H * L,
SEQ_LEN]</param>
 /// <returns>Выходной градиент по входу dL/dX [B * H * L, SEQ_LEN]</returns>
 public Tensor SoftmaxBackward(Tensor gradOutput, Tensor y)
 {
    int seqLen = y.Shape[y.Shape.Length - 1];
    int numRows = y.Size / seqLen;

    // Выделяем тензор для результирующего градиента по входу (по форме совпадает с Y)
    Tensor gradInput = new Tensor(y.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: false);

    // Настройка сетки потоков (идентично forward шагу)
    IntPtr localSize = new IntPtr(256);
    IntPtr globalSize = new IntPtr(numRows * 256);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
    int t = 0;
    // Передаем аргументы в Си-ядро softmax_backward:
    // kernel void softmax_backward(global const float* grad_output, global const float* Y, global
float* grad_input, const int seq_len)
    _kernelSoftmaxBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
    _kernelSoftmaxBackward.Arguments[t++].SetValue(y.DataBuffer);
    _kernelSoftmaxBackward.Arguments[t++].SetValue(gradInput.DataBuffer);
    _kernelSoftmaxBackward.Arguments[t++].SetValue(seqLen);

    // Запуск ядра
    activeQueue.EnqueueNDRangeKernel(_kernelSoftmaxBackward, null, new[] { globalSize },
new[] { localSize });
    activeQueue.Finish();
```

```
    return gradInput;
}
#region RSNORM
// Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
private Kernel _kernelRmsNormForward;
private Kernel _kernelRmsNormBackward;

// Добавьте их инициализацию в метод LoadKernels():
// _kernelRmsNormForward = _prgRmsNorm.CreateKernel("rmsnorm_forward");
// _kernelRmsNormBackward = _prgRmsNorm.CreateKernel("rmsnorm_backward");

/// <summary>
/// Глобальная операция RMSNorm (Forward).
/// Обрабатывает тензор любой формы, выполняя нормализацию по последней оси d_model.
/// </summary>
public Tensor RmsNorm(Tensor x, Tensor gamma)
{
    int dModel = x.Shape[x.Shape.Length - 1]; // Последняя ось
    int numRows = x.Size / dModel;           // Количество строк (B * L)

    // Выходной тензор
    Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad:
x.RequiresGrad || gamma.RequiresGrad);

    // Буфер для кэша инвертированного RMS (нужен для Backward)
    // Размер: numRows * sizeof(float)
    NOpenCL.Buffer invRmsCache =
_computeCtx.Context.CreateBuffer(MemoryFlags.ReadWrite, numRows * sizeof(float));

    // Настройка сетки потоков
    IntPtr localSize = new IntPtr(256);
    IntPtr globalSize = new IntPtr(numRows * 256);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
    int t = 0;
    // Передаем аргументы ядра:
    // kernel void rmsnorm_forward(global const float* X, global const float* gamma, global float*
Y, global float* inv_rms_cache, const int d_model)
    _kernelRmsNormForward.Arguments[t++].SetValue(x.DataBuffer);
    _kernelRmsNormForward.Arguments[t++].SetValue(gamma.DataBuffer);
    _kernelRmsNormForward.Arguments[t++].SetValue(y.DataBuffer);
    _kernelRmsNormForward.Arguments[t++].SetValue(invRmsCache);
    _kernelRmsNormForward.Arguments[t++].SetValue(dModel);

    activeQueue.EnqueueNDRangeKernel(_kernelRmsNormForward, null, new[] { globalSize },
new[] { localSize });
    activeQueue.Finish();

    // Если обучаемся, сохраняем кэш слоя в GradientStorage
    if (y.RequiresGrad)
    {
```

```
            var buffers = new LayerBuffers();
            buffers.layerBuffers.Add("inv_rms_cache", invRmsCache);
            GradientStorage.Push(buffers);

            // Привязываем узел операции к графу
            y.Creator = new RMSNormOp(x, gamma, y, this);
        }
        else
        {
            // Если обучение не идет, кэш нам больше не нужен, сразу освобождаем VRAM
            invRmsCache.Dispose();
        }

        return y;
    }

    /// <summary>
    /// Обратный проход глобальной операции RMSNorm.
    /// Вычисляет градиенты по входу X и по весам Gamma.
    /// </summary>
    public unsafe (Tensor gradX, Tensor gradGamma) RmsNormBackward(Tensor gradOutput, Tensor
x, Tensor gamma, NOpenCL.Buffer invRmsCache)
    {
        int dModel = x.Shape[x.Shape.Length - 1];
        int numRows = x.Size / dModel;

        // Выделяем выходные градиенты
        Tensor gradX = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: false);
        Tensor gradGamma = new Tensor(gamma.Shape, _computeCtx.Context,
MemoryFlags.ReadWrite, requiresGrad: false);

        // Промежуточный буфер градиентов по весам для каждой строки (для последующей
редукции)
        long partSizeInBytes = (long)x.Size * sizeof(float);
        using NOpenCL.Buffer gradGammaPart =
_computeCtx.Context.CreateBuffer(MemoryFlags.ReadWrite, partSizeInBytes);

        IntPtr localSize = new IntPtr(256);
        IntPtr globalSize = new IntPtr(numRows * 256);

        CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
        int t = 0;
        // Передаем аргументы ядра:
        // kernel void rmsnorm_backward(global const float* grad_output, global const float* X, global
const float* gamma, global const float* inv_rms_cache, global float* grad_input, global float*
grad_gamma_part, const int d_model)
        _kernelRmsNormBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(x.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(gamma.DataBuffer);
        _kernelRmsNormBackward.Arguments[t++].SetValue(invRmsCache);
        _kernelRmsNormBackward.Arguments[t++].SetValue(gradX.DataBuffer);
```

```
    _kernelRmsNormBackward.Arguments[t++].SetValue(gradGammaPart);
    _kernelRmsNormBackward.Arguments[t++].SetValue(dModel);

    activeQueue.EnqueueNDRangeKernel(_kernelRmsNormBackward, null, new[] { globalSize },
new[] { localSize });
    activeQueue.Finish();

    // --- РЕДУКЦИЯ ГРАДИЕНТОВ ПО ВЕСАМ (Адаптация кода с numpy на C#) ---
    // Считываем частичные градиенты во временный массив на CPU для суммирования строк
    float[] hostParts = new float[numRows * dModel];
    fixed (float* hp = hostParts)
    {
       activeQueue.EnqueueReadBuffer(gradGammaPart, true, 0, hostParts.Length * sizeof(float),
(IntPtr)hp);
    }
    activeQueue.Finish();

    // Суммируем градиенты от всех токенов/строк в единый вектор весов Gamma
    float[] finalGradGamma = new float[dModel];
    for (int r = 0; r < numRows; r++)
    {
       int rowOffset = r * dModel;
       for (int c = 0; c < dModel; c++)
       {
          finalGradGamma[c] += hostParts[rowOffset + c];
       }
    }

    // Загружаем финальный саккумулированный вектор градиента обратно во VRAM буфер
gradGamma
    fixed (float* fgg = finalGradGamma)
    {
       activeQueue.EnqueueWriteBuffer(gradGamma.DataBuffer, true, 0, finalGradGamma.Length
* sizeof(float), (IntPtr)fgg);
    }
    activeQueue.Finish();

    return (gradX, gradGamma);
 }

 #endregion
 #region Permute
 // Добавьте эту переменную в начало класса NnClGlob к остальным ядрам:
 private Kernel _kernelPermute;

 // Добавьте её инициализацию в метод LoadKernels():
 // _kernelPermute = _prgPermute.CreateKernel("tensor_permute");

 /// <summary>
 /// Глобальная перестановка осей тензора (Аналог x.permute(...) или x.transpose(...)).
 /// Поддерживает тензоры любой размерности.
 /// </summary>
```

```
 public unsafe Tensor Permute(Tensor x, int[] axes)
 {
    if (axes.Length != x.Shape.Length)
       throw new ArgumentException("Количество осей перестановки должно совпадать с размерностью тензора.");

    // Вычисляем новую форму тензора на основе массива перестановки осей
    int[] dstShape = new int[axes.Length];
    for (int i = 0; i < axes.Length; i++)
    {
       dstShape[i] = x.Shape[axes[i]];
    }

    Tensor y = new Tensor(dstShape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: x.RequiresGrad);

    // Подготовка метаданных для Си-ядра: размеры осей во VRAM буферы int32
    int[] srcDims = x.Shape;
    int[] dstDims = dstShape;
    fixed (int* ax = axes, srd = srcDims, dstd = dstDims)
    {
       Context ctx = _computeCtx.Context;
       using NOpenCL.Buffer srcDimsBuf = ctx.CreateBuffer(MemoryFlags.ReadOnly | MemoryFlags.UseHostPointer, srcDims.Length * sizeof(int), (IntPtr)srd);
       using NOpenCL.Buffer dstDimsBuf = ctx.CreateBuffer(MemoryFlags.ReadOnly | MemoryFlags.UseHostPointer, dstDims.Length * sizeof(int), (IntPtr)dstd);
       using NOpenCL.Buffer axesBuf = ctx.CreateBuffer(MemoryFlags.ReadOnly | MemoryFlags.UseHostPointer, axes.Length * sizeof(int), (IntPtr)ax);

       IntPtr globalSize = new IntPtr(x.Size);
       CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
       int t = 0;
       // Передаем аргументы в Си-ядро tensor_permute:
       // kernel void tensor_permute(global const float* src, global float* dst, global const int* src_dims, global const int* dst_dims, global const int* permute_axes, const int num_dims, const int total_elements)
       _kernelPermute.Arguments[t++].SetValue(x.DataBuffer);
       _kernelPermute.Arguments[t++].SetValue(y.DataBuffer);
       _kernelPermute.Arguments[t++].SetValue(srcDimsBuf);
       _kernelPermute.Arguments[t++].SetValue(dstDimsBuf);
       _kernelPermute.Arguments[t++].SetValue(axesBuf);
       _kernelPermute.Arguments[t++].SetValue(x.Shape.Length);
       _kernelPermute.Arguments[t++].SetValue(x.Size);

       activeQueue.EnqueueNDRangeKernel(_kernelPermute, new[] { globalSize }, null);
       activeQueue.Finish();
    }
    return y;
 }

 #endregion
 #region BMM
```

```
// Добавьте эту переменную в начало класса NnClGlob к остальным ядрам:
private Kernel _kernelBmmTiled;

// Добавьте её инициализацию в метод LoadKernels():
// _kernelBmmTiled = _prgMatMul.CreateKernel("bmm_tiled");

/// <summary>
/// Пакетное матричное умножение (Batch Matrix Multiplication).
/// Ожидает строго 3D-тензоры: A [Batch, M, K], B [Batch, K, N] -> Выход [Batch, M, N].
/// </summary>
/// <summary>
/// Пакетное матричное умножение с поддержкой Batch Broadcasting для весов модели.
/// </summary>
public Tensor Bmm(Tensor a, Tensor b)
{
    if (a.Shape.Length != 3 || b.Shape.Length != 3)
        throw new ArgumentException("BMM ожидает строго 3D-тензоры формы [Batch, M, K].");

    // ПРАВИЛО BROADCASTING ДЛЯ БАТЧА:
    // Если батч у B равен 1, а у A больше 1, то разрешаем операцию (веса применятся ко всем примерам)
    if (a.Shape[0] != b.Shape[0] && b.Shape[0] != 1)
        throw new ArgumentException($"Несовпадение батч-размерностей: {a.Shape[0]} и {b.Shape[0]}.");

    if (a.Shape[2] != b.Shape[1])
        throw new ArgumentException($"Несовпадение внутренних осей матриц: K-ось {a.Shape[2]} != {b.Shape[1]}.");

    int batch = a.Shape[0]; // Итоговый батч всегда равен батчу данных (3)
    int m = a.Shape[1];
    int k = a.Shape[2];
    int n = b.Shape[2];

    int[] outShape = { batch, m, n };
    Tensor c = new Tensor(outShape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: a.RequiresGrad || b.RequiresGrad);

    int batchStrideA = m * k;

    // ВАЖНОЕ АРХИТЕКТУРНОЕ ИСПРАВЛЕНИЕ:
    // Если у тензора B (веса) батч равен 1, то шаг смещения пакета во VRAM принудительно зануляется!
    // Ядро на GPU будет читать одну и ту же матрицу для каждого шага батча
    int batchStrideB = (b.Shape[0] == 1) ? 0 : (k * n);

    int batchStrideC = m * n;

    // Настройка 3D-сетки потоков под тайлы 16x16
    int gx = (int)Math.Ceiling(n / 16.0) * 16;
    int gy = (int)Math.Ceiling(m / 16.0) * 16;
```

```
    int gz = batch;

    IntPtr localSizeX = new IntPtr(16);
    IntPtr localSizeY = new IntPtr(16);
    IntPtr localSizeZ = new IntPtr(1);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
    int t = 0;
    _kernelBmmTiled.Arguments[t++].SetValue(a.DataBuffer);
    _kernelBmmTiled.Arguments[t++].SetValue(b.DataBuffer);
    _kernelBmmTiled.Arguments[t++].SetValue(c.DataBuffer);
    _kernelBmmTiled.Arguments[t++].SetValue(m);
    _kernelBmmTiled.Arguments[t++].SetValue(k);
    _kernelBmmTiled.Arguments[t++].SetValue(n);
    _kernelBmmTiled.Arguments[t++].SetValue(batchStrideA);
    _kernelBmmTiled.Arguments[t++].SetValue(batchStrideB); // Передаем скорректированный
шаг!
    _kernelBmmTiled.Arguments[t++].SetValue(batchStrideC);

    activeQueue.EnqueueNDRangeKernel(
        _kernelBmmTiled,
        null,
        new[] { new IntPtr(gx), new IntPtr(gy), new IntPtr(gz) },
        new[] { localSizeX, localSizeY, localSizeZ }
    );
    activeQueue.Finish();

    if (c.RequiresGrad)
    {
        c.Creator = new BmmOp(a, b, c, this);
    }

    return c;
 }


 #endregion
 #region CasualConv1d
 // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
 private Kernel _kernelConvForward;
 private Kernel _kernelConvBackwardInput;
 private Kernel _kernelConvBackwardWeights;

 // Добавьте их инициализацию в метод LoadKernels():
 // _kernelConvForward = _prgConv.CreateKernel("causal_conv1d_depthwise");
 // _kernelConvBackwardInput = _prgConv.CreateKernel("causal_conv1d_backward_input");
 // _kernelConvBackwardWeights = _prgConv.CreateKernel("causal_conv1d_backward_weights");

 /// <summary>
 /// Каузальная одномерная Depthwise свёртка (Forward).
 /// Форма X: [Batch, Channels, Length]
 /// Форма Weight: [Channels, KernelSize]
```

```
/// Форма Bias: [Channels]
/// </summary>
public Tensor CausalConv1d(Tensor x, Tensor weight, Tensor bias)
{
    if (x.Shape.Length != 3)
        throw new ArgumentException("Входной тензор X должен иметь 3 измерения [Batch, Channels, Length].");

    int batch = x.Shape[0];
    int channels = x.Shape[1];
    int length = x.Shape[2];
    int kernelSize = weight.Shape[1];

    Tensor y = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite,
        requiresGrad: x.RequiresGrad || weight.RequiresGrad || bias.RequiresGrad);

    // Настройка 3D сетки потоков по осям временной последовательности, каналов и батча
    IntPtr globalSizeX = new IntPtr(length);
    IntPtr globalSizeY = new IntPtr(channels);
    IntPtr globalSizeZ = new IntPtr(batch);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
    int t = 0;
    // Передаем аргументы ядра causal_conv1d_depthwise
    _kernelConvForward.Arguments[t++].SetValue(x.DataBuffer);
    _kernelConvForward.Arguments[t++].SetValue(weight.DataBuffer);
    _kernelConvForward.Arguments[t++].SetValue(bias.DataBuffer);
    _kernelConvForward.Arguments[t++].SetValue(y.DataBuffer);
    _kernelConvForward.Arguments[t++].SetValue(batch);
    _kernelConvForward.Arguments[t++].SetValue(channels);
    _kernelConvForward.Arguments[t++].SetValue(length);
    _kernelConvForward.Arguments[t++].SetValue(kernelSize);

    activeQueue.EnqueueNDRangeKernel(
        _kernelConvForward,

        new[] { globalSizeX, globalSizeY, globalSizeZ },
        null
    );
    activeQueue.Finish();

    if (y.RequiresGrad)
    {
        y.Creator = new CausalConv1dOp(x, weight, bias, y, this);
    }

    return y;
}

/// <summary>
/// Обратный проход каузальной свёртки. Вычисляет градиенты по входу, весам и смещению.
/// </summary>
```

```
 public (Tensor gradX, Tensor gradWeight, Tensor gradBias) CausalConv1dBackward(Tensor gradOutput, Tensor x, Tensor weight)
 {
    int batch = x.Shape[0];
    int channels = x.Shape[1];
    int length = x.Shape[2];
    int kernelSize = weight.Shape[1];

    // Аллокация результирующих тензоров градиентов
    Tensor gradX = new Tensor(x.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);
    Tensor gradWeight = new Tensor(weight.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);
    Tensor gradBias = new Tensor(new[] { channels }, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

    // 1. Запуск ядра расчета градиента по входу X (Сетка 3D: Length, Channels, Batch)
    IntPtr gSizeInX = new IntPtr(length);
    IntPtr gSizeInY = new IntPtr(channels);
    IntPtr gSizeInZ = new IntPtr(batch);
    int t = 0;
    _kernelConvBackwardInput.Arguments[t++].SetValue(gradOutput.DataBuffer);
    _kernelConvBackwardInput.Arguments[t++].SetValue(weight.DataBuffer);
    _kernelConvBackwardInput.Arguments[t++].SetValue(gradX.DataBuffer);
    _kernelConvBackwardInput.Arguments[t++].SetValue(batch);
    _kernelConvBackwardInput.Arguments[t++].SetValue(channels);
    _kernelConvBackwardInput.Arguments[t++].SetValue(length);
    _kernelConvBackwardInput.Arguments[t++].SetValue(kernelSize);

    activeQueue.EnqueueNDRangeKernel(
       _kernelConvBackwardInput,

       new[] { gSizeInX, gSizeInY, gSizeInZ },
       null
    );

    // 2. Запуск ядра расчета градиентов по параметрам Weight и Bias (Сетка 2D: KernelSize, Channels)
    IntPtr gSizeW_X = new IntPtr(kernelSize);
    IntPtr gSizeW_Y = new IntPtr(channels);
    t=0;
    _kernelConvBackwardWeights.Arguments[t++].SetValue(gradOutput.DataBuffer);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(x.DataBuffer);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(gradWeight.DataBuffer);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(gradBias.DataBuffer);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(batch);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(channels);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(length);
    _kernelConvBackwardWeights.Arguments[t++].SetValue(kernelSize);
```

```
    activeQueue.EnqueueNDRangeKernel(
        _kernelConvBackwardWeights,

        new[] { gSizeW_X, gSizeW_Y },
        null
    );

    // Дожидаемся окончания работы обоих ядер на GPU
    activeQueue.Finish();

    return (gradX, gradWeight, gradBias);
 }

 #endregion
 #region Embedding
 // Добавьте эти переменные в начало класса NnClGlob к остальным ядрам:
 private Kernel _kernelEmbeddingForward;
 private Kernel _kernelEmbeddingBackward;

 // Добавьте их инициализацию в метод LoadKernels():
 // _kernelEmbeddingForward = _prgEmbedding.CreateKernel("embedding_forward");
 // _kernelEmbeddingBackward = _prgEmbedding.CreateKernel("embedding_backward");

 /// <summary>
 /// Выборка эмбеддингов по ID токенов (Forward).
 /// </summary>
 /// <param name="xTokensBuf">Низкоуровневый OpenCL буфер, хранящий целочисленные ID
токенов (int32).</param>
 /// <param name="weight">Тензор весов эмбеддингов формы [VocabSize,
EmbeddingDim].</param>
 /// <param name="totalTokens">Общее количество токенов в батче (Batch * SeqLen).</param>
 /// <returns>Выходной тензор формы [Batch * SeqLen, EmbeddingDim] или эквивалентной
многомерной формы.</returns>
 public Tensor Embedding(NOpenCL.Buffer xTokensBuf, Tensor weight, int totalTokens)
 {
    if (weight.Shape.Length != 2)
        throw new ArgumentException("Матрица весов Embedding должна быть строго 2D
тензором [VocabSize, EmbeddingDim].");

    int vocabSize = weight.Shape[0];
    int embeddingDim = weight.Shape[1];

    // Создаем выходной тензор. Форма: [TotalTokens, EmbeddingDim]
    int[] outShape = { totalTokens, embeddingDim };
    Tensor y = new Tensor(outShape, _computeCtx.Context, MemoryFlags.ReadWrite,
requiresGrad: weight.RequiresGrad);

    // Настройка 2D-сетки потоков: по количеству токенов и по размерности эмбеддинга
    IntPtr globalSizeX = new IntPtr(totalTokens);
    IntPtr globalSizeY = new IntPtr(embeddingDim);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();
```

```
    int t = 0;
    // Передаем аргументы в Си-ядро embedding_forward:
    // kernel void embedding_forward(global const int* X_tokens, global const float* weight, global float* Y_out, const int embedding_dim, const int total_tokens)
    _kernelEmbeddingForward.Arguments[t++].SetValue(xTokensBuf);
    _kernelEmbeddingForward.Arguments[t++].SetValue(weight.DataBuffer);
    _kernelEmbeddingForward.Arguments[t++].SetValue(y.DataBuffer);
    _kernelEmbeddingForward.Arguments[t++].SetValue(embeddingDim);
    _kernelEmbeddingForward.Arguments[t++].SetValue(totalTokens);

    activeQueue.EnqueueNDRangeKernel(
        _kernelEmbeddingForward,

        new[] { globalSizeX, globalSizeY },
        null
    );
    activeQueue.Finish();

    // Если матрица весов требует градиентов, регистрируем операцию в графе
    if (y.RequiresGrad)
    {
        y.Creator = new EmbeddingOp(xTokensBuf, weight, y, totalTokens, this);
    }

    return y;
 }

 /// <summary>
 /// Обратный проход слоя Embedding. Рассчитывает градиент по матрице весов (gradWeight).
 /// Альтернативная реализация: обнуление буфера через запись host-массива (замена отсутствующего FillBuffer).
 /// </summary>
 public unsafe Tensor EmbeddingBackward(Tensor gradOutput, NOpenCL.Buffer xTokensBuf, Tensor weight, int totalTokens)
 {
    int vocabSize = weight.Shape[0];
    int embeddingDim = weight.Shape[1];

    // Выделяем тензор для накопления градиентов по весам (форма совпадает с Weight)
    Tensor gradWeight = new Tensor(weight.Shape, _computeCtx.Context, MemoryFlags.ReadWrite, requiresGrad: false);

    CommandQueue activeQueue = DeviceQueueRegistry.GetCurrent();

    //
    // ИСПРАВЛЕНИЕ: ОБНУЛЕНИЕ БУФЕРА ЧЕРЕЗ ХОСТ-МАССИВ СИЛАМИ .NET 10
    //
    // Создаем массив нулей на CPU. В C# при выделении массива float[]
```

```
    // среда выполнения .NET гарантирует, что он изначально полностью заполнен нулями.
    float[] zeroHostPattern = new float[gradWeight.Size];

    // Вычисляем размер буфера в байтах
    long bufferSizeInBytes = (long)gradWeight.Size * sizeof(float);
    fixed (float* zhp = zeroHostPattern)
    {
        // Синхронно записываем массив нулей во VRAM девайса, чтобы очистить старые данные
        // Аргументы: буфер, blocking_write (true), смещение, размер в байтах, массив-источник
        activeQueue.EnqueueWriteBuffer(gradWeight.DataBuffer, true, 0, bufferSizeInBytes, (IntPtr)zhp);
    }
    //============================================================================

    // Настройка 2D-сетки потоков под конфигурацию обратного прохода
    IntPtr globalSizeX = new IntPtr(totalTokens);
    IntPtr globalSizeY = new IntPtr(embeddingDim);
    int t = 0;
    // Передаем аргументы в Си-ядро embedding_backward:
    // kernel void embedding_backward(global const float* grad_output, global const int* X_tokens, global float* grad_weight, const int embedding_dim, const int total_tokens)
    _kernelEmbeddingBackward.Arguments[t++].SetValue(gradOutput.DataBuffer);
    _kernelEmbeddingBackward.Arguments[t++].SetValue(xTokensBuf);
    _kernelEmbeddingBackward.Arguments[t++].SetValue(gradWeight.DataBuffer);
    _kernelEmbeddingBackward.Arguments[t++].SetValue(embeddingDim);
    _kernelEmbeddingBackward.Arguments[t++].SetValue(totalTokens);

    // Запускаем атомарное ядро
    activeQueue.EnqueueNDRangeKernel(
        _kernelEmbeddingBackward,

        new[] { globalSizeX, globalSizeY },
        null
    );
    activeQueue.Finish();

    return gradWeight;
 }
```

Вот примеры для глобальных объектов:

```
public Tensor Forward(Tensor x)
{
    // 1. Слой нормализации (теперь RmsGamma гарантированно состоит из 1.0f)
    Tensor normalized = NnClGlob.RmsNorm(x, RmsGamma);

    // 2. Линейная проекция через пакетное умножение матриц
    Tensor projected = NnClGlob.Bmm(normalized, ProjectionWeights);

    // ИНТЕГРАЦИЯ: Безопасно скачиваем чистые данные проекции на хост (CPU)
    ProjectedHost = HostReturner.ToCpu(projected);

    // 3. Вычисление вероятностей распределения внимания
    Tensor probabilities = NnClGlob.Softmax(projected, dim: -1);

    return probabilities;
}


/// <summary>
/// Выполняет высокооптимизированный прямой проход: BMM -> JIT Сложение с Bias.
/// На вход ожидает 3D тензор данных формы [Batch, M, InFeatures].
/// </summary>
public Tensor Forward(Tensor x)
{
    if (x.Shape[x.Shape.Length - 1] != _inFeatures)
    {
        throw new ArgumentException($"Несоответствие размерностей входа.
Ожидалось {_inFeatures} признаков, получено {x.Shape[x.Shape.Length - 1]}.");
    }

    // ШАГ 1 (ГЛОБАЛЬНЫЙ): Тяжелое матричное умножение X x W на GPU
    // Выход projected получит форму [Batch, M, OutFeatures]
    Tensor projected = NnClGlob.Bmm(x, Weight);

    // ШАГ 2 (ЛОКАЛЬНЫЙ JIT): Сплавление операции прибавления Bias на регистрах
GPU.
    // Передаем projected и bias в наш Roslyn-компонент
    var tensorArgs = new Dictionary<string, Tensor> {
        { "P", projected },
        { "b", Bias }
    };

    // Математический псевдокод для JIT.
    // Благодаря нашему модулю Broadcasting, вектор 'b' автоматически прибавится
к каждой строке матрицы 'P'!
    string pseudoCode = "P = P + b;";

    // Вызываем RunLocal, который вернет НОВЫЙ чистый тензор без затирания графа
Autograd
    Dictionary<string, Tensor> localOutputs = NnClGlob.RunLocal(pseudoCode,
tensorArgs, constants: null);
    Tensor output = localOutputs["P"];

    return output;
}
```

! Сейчас broadcast матриц работает не совсем корректно, поэтому используй размеры тензоров не требующих broadcast!